



Universidad Nacional del Nordeste



Facultad de Ciencias Exactas, Naturales y Agrimensura

Licenciatura en Sistemas de Información

Cátedra: ***Base de Datos II***

Año: 2020

GRUPO 7

• Díaz, Adela	CU: 49.581
• Fernández, Paulina Belén	CU: 51.417
• Gómez, Mario Andrés	CU: 42.291
• Ledezma, Lucía del Valle	CU: 43.293
• Pruyas, Araceli de los Angeles Itati	CU: 52.323
• Vallejos, Santajuliana Giuliano	CU: 52.643

Monografía:

Técnicas de Control de la Concurrency

Índice

Índice de Figuras	3
1. Introducción.....	4
1.2 Control de la concurrencia y transacciones.....	5
1.3 Porqué es necesario el control de la concurrencia.....	6
2. Técnicas de bloqueo para el control de concurrencia.	7
2.1 Candados Binarios	7
2.2 Candados Comparativos y Exclusivos.....	8
2.3 Conversión de bloqueos.....	10
2.4 Protocolo de bloqueo de dos fases.....	11
2.5 Interbloqueos e inanición	12
2.6 Protocolos de prevención de interbloqueos.....	12
2.7 Detección del interbloqueo.....	13
2.8 Tiempos limitados.....	14
2.9 Inanición (espera indefinida)	14
3. Control de la concurrencia basado en la ordenación de marcas de tiempo.	15
3.1 Marcas de tiempo	15
3.1.1 Algoritmo de ordenación de marcas de tiempo	15
3.1.2 Ordenación de marcas de tiempo (TO, Timestamp Ordering) básica.	15
3.1.3 Ordenación de marcas de tiempo (TO, Timestamp Ordering) estricta	16
4. Técnicas multiversión para controlar la concurrencia.....	17
4.1 Técnica multiversión basada en la ordenación de las marcas de tiempo.....	17
4.2 Bloqueo en dos fases multiversión utilizando bloqueos de certificación	17
5. Técnicas de control de la concurrencia optimistas (validación)	19
6. Granularidad de los elementos de datos y bloqueo de la granularidad múltiple	20
6.1 Bloqueo por nivel de granularidad múltiple.....	21
7. Uso de bloqueos para controlar la concurrencia en los índices.	22
8. Otros problemas del control de la concurrencia	24
8.1 Inserción, eliminación y registros fantasmas	24
8.2 Transacciones interactivas.....	24
8.3 Cerrojos	25
9. Conclusión.....	26
Bibliografía:.....	27

Índice de Figuras

Figura 1. Operaciones de bloquear y desbloquear para candados binarios [3].	8
Figura 2. Operaciones de bloqueo y desbloqueo para candados de dos modos (leer_escribir o compartido_exclusivo) [3].	10
Figura 3. Tabla de compatibilidad para el esquema de bloqueo [3].	18
Figura 4. Tabla de compatibilidad para el esquema de bloqueo de lectura/escritura/certificación [3].	18
Figura 5. Protocolo de nivel de granularidad múltiple [3].	21

1. Introducción

Se define *transacción* en un sistema de gestión de base de datos (SGBD) como un conjunto de operaciones que se realizan como una sola unidad lógica de trabajo.

Dicha unidad debe cumplir con cuatro propiedades básicas, conocidas como *atomicidad*, *coherencia*, *aislamiento* y *durabilidad* (ACID), para poder ser clasificada como una transacción [1].

Los SGBD incluyen *protocolos* para asegurar la propiedad de aislamiento de transacciones que se ejecutan concurrentemente.

La mayoría de las técnicas de control de concurrencia aseguran que los planes sean seriables. Estas técnicas se ejecutan conjuntamente con las técnicas de recuperación. [2].

Un importante conjunto de protocolos emplea la técnica de bloquear (mediante un candado) elementos de información para evitar que más de una transacción tenga acceso a los elementos al mismo tiempo, otro conjunto de protocolos utiliza marcas de tiempo de transacciones. Una *marca de tiempo* es un identificador único generado por el sistema para cada transacción.

También existen protocolos de control de concurrencia de multiversión, que emplean múltiples versiones de un elemento de información.

Los protocolos basados en el concepto de validación o certificación de una transacción después de que ejecuta sus operaciones, suelen recibir el nombre de protocolos optimistas.

Otro factor que afecta el control de concurrencia es la granularidad de los elementos, esto es, qué porción de la base de datos representa un elemento. Un elemento puede ser tan pequeño como un solo valor o tan grande como un bloque de disco, un archivo o incluso la base de datos entera [3].

1.2 Control de la concurrencia y transacciones

El control de las transacciones debe residir en las aplicaciones, primordialmente al especificar el inicio y fin de las mismas, en puntos que aseguren la coherencia lógica de los datos. Por lo tanto, se deben explicitar las secuencias de modificaciones de datos que los dejan en un estado coherente en relación a las reglas de negocios de la organización [1].

Lo expuesto se logra cumpliendo con las propiedades ACID. A continuación, se sintetizan cada una de ellas [5]:

- **Atomicidad:** una transacción debe ser una unidad atómica de trabajo, tanto si se realizan todas sus modificaciones en los datos, como si no se realiza ninguna de ellas.
- **Coherencia:** cuando finaliza una transacción, debe dejar todos los datos en un estado coherente. En una base de datos relacional, se deben aplicar todas las reglas a las modificaciones de la transacción para mantener la integridad de todos los datos. Todas las estructuras internas de datos, como índices de árbol b o listas doblemente vinculadas, deben estar correctas al final de la transacción.
- **Aislamiento:** las modificaciones realizadas por transacciones simultáneas se deben aislar de las modificaciones llevadas a cabo por otras transacciones simultáneas. Una transacción reconoce los datos en el estado en que estaban antes de que otra transacción simultánea los modificara o después de que la segunda transacción haya concluido, pero no reconoce un estado intermedio. Esto se conoce como *seriabilidad*, ya que deriva en la capacidad de volver a cargar los datos iniciales y reproducir una serie de transacciones para finalizar con los datos en el mismo estado en que estaban después de realizar las transacciones originales.
- **Durabilidad:** una vez concluida una transacción, sus efectos son permanentes en el sistema. Las modificaciones persisten aún en el caso de producirse un error del sistema.

Vemos también la necesidad de introducir algunos conceptos básicos [3]:

Plan: orden de ejecución de las operaciones de varias transacciones que se ejecutan de manera concurrente e intercalada. Un plan P de n transacciones $T_1, T_2, \dots, T_n$ es un ordenamiento para las operaciones de las transacciones de modo que:

- Para cada transacción T_i que participa en P , las operaciones de T_i deben aparecer en el mismo orden que ocurren en T_i .
- En P las operaciones de T_k pueden intercalarse con T_i .

Plan en serie: un plan P es *en serie* si, por cada transacción P que participa en el plan, todas las operaciones de T se ejecutan consecutivamente en el plan.

Plan serializable: un plan P es *serializable* si es equivalente a algún plan en serie de las mismas n transacciones. Cabe señalar, que las operaciones de otras transacciones T_i , se pueden intercalar con las operaciones de T_i en P .

En un SGDB multiusuario, los elementos de información almacenados son los recursos primarios a los que pueden tener acceso concurrente los programas de usuarios, que constantemente obtienen información de la base de datos y la modifican. La ejecución de un programa que lee o modifica el contenido de una base de datos es lo que llamamos *transacción* [3].

1.3 Porqué es necesario el control de la concurrencia

Pueden surgir varios problemas cuando las transacciones concurrentes se ejecutan de manera no controlada [3]:

El problema de la actualización perdida. Ocurre cuando dos transacciones que tienen acceso a los mismos elementos de la base de datos tienen sus operaciones intercaladas de modo que hacen incorrecto el valor de algún elemento.

El problema de la actualización temporal (o lectura sucia). Esto ocurre cuando una transacción actualiza un elemento de la base de datos y luego la transacción falla por alguna razón. Otra transacción tiene acceso al elemento actualizado antes de que se restaure a su valor original.

Problema del resumen incorrecto. Si una transacción está calculando una función agregada de resumen sobre varios registros mientras otras transacciones están actualizando algunos de ellos, puede ser que la función agregada calcule algunos valores antes de que se actualicen y otros después de actualizarse.

Lectura no repetible. En el que una transacción T lee un elemento dos veces y otra transacción T' modifica el elemento entre las dos lecturas. Así, T recibe diferentes valores en sus dos lecturas del mismo elemento.

Es necesario que el sistema controle la interacción entre las transacciones concurrentes, el mismo, se lleva a cabo a través de uno de los muchos mecanismos existentes llamado esquemas de control de concurrencia [6].

2. Técnicas de bloqueo para el control de concurrencia.

Una de las principales técnicas para controlar la ejecución concurrente de las transacciones se basa en el concepto de bloquear elementos de información. Un *candado* es una variable asociada a un elemento de información de la base de datos que describe el estado de ese elemento respecto a las posibles operaciones que se pueden aplicar a él.

Se utilizan los candados como una forma de sincronizar el acceso a los elementos de la base de datos por parte de transacciones concurrentes.

Una forma de asegurar la secuencialidad es exigir que el acceso a los elementos de datos se haga en exclusión mutua, es decir, mientras una transacción accede a un elemento de datos, ninguna otra transacción puede modificar dicho elemento. El método más habitual que se usa para implementar este requisito es permitir que una transacción acceda a un elemento de datos sólo si posee actualmente un bloqueo sobre dicho elemento.

Se puede utilizar varios tipos de candados en el control de concurrencia.

2.1 Candados Binarios

Un candado binario puede tener dos estados o valores: *bloqueado* y *desbloqueado* (1 y 0). A cada elemento X de la base de datos se asocia un candado distinto. Si el valor del candado sobre X es 1, ninguna operación de base de datos que solicite el elemento podrá tener acceso a él. Si el valor del candado sobre X es 0, se podrá tener acceso al elemento cuando se solicite. $CANDADO(X)$ será el valor del candado asociado al elemento X .

Cuando se usa bloqueo binario se debe incluir en las transacciones dos operaciones, *bloquear_elemento* y *desbloquear_elemento*. Una transacción solicita acceso a un elemento X emitiendo una operación *bloquear_elemento*(X) si $CANDADO(X) = 1$, la transacción tiene que esperar; si no, la transacción asigna 1 a $CANDADO(X)$ (bloquea el elemento) y obtiene el acceso. Cuando la transacción termina de usar el elemento, emite una operación *desbloquear_elemento*(X), que asigna 0 a $CANDADO(X)$ (desbloquea el elemento) para que otras transacciones puedan tener acceso a X . Así, un candado binario impone una exclusión mutua sobre el elemento de información.

El SGBD cuenta con un subsistema gestor de bloqueo para mantenerse al tanto del acceso a los candados y controlarlo.

Cuando se utiliza el esquema de bloqueo binario, toda transacción debe obedecer las siguientes reglas:

1. Una transacción T debe emitir la operación `bloquear_elemento(X)` antes de que se realice cualquier operación `leer_elemento(X)` o `escribir_elemento(X)` en T .
2. Una transacción T debe emitir la operación `desbloquear_elemento(X)` después de haber completado todas las operaciones `leer_elemento(X)` y `escribir_elemento(X)` en T .
3. Una transacción T no emitirá una operación `bloquear_elemento(X)` si ya posee el bloqueo del elemento X .
4. Una transacción T no emitirá una operación `desbloquear_elemento(X)` a menos que ya posea el bloqueo del elemento X .

```

bloquear_elemento(X):
  B: si CANDADO(X)=0 (* el elemento está desbloqueado *)
    entonces CANDADO(X) ← 1 (* bloquear el elemento *)
    de otro modo comenzar
      esperar (hasta que CANDADO(X)=0 y
        el gestor de bloqueo despierte la transacción);
    ir a B
  fin;

desbloquear_elemento(X):
  CANDADO(X) ← 0 (* desbloquear el elemento *);
  si alguna transacción está esperando
    entonces despertar una de las transacciones que esperan;

```

Figura 1. Operaciones de bloquear y desbloquear para candados binarios [3].

2.2 Candados Comparativos y Exclusivos

El esquema de bloqueo binario es demasiado restrictivo en lo general porque, como máximo, una transacción puede poseer un candado sobre un elemento dado. Se debe permitir que varias transacciones tengan acceso al mismo elemento X si lo hacen exclusivamente para leerlo. Sin embargo, si una transacción va a escribir un elemento X deberá poseer acceso exclusivo a él. Para este fin, podemos usar un tipo de candado diferente llamado *candado de modo múltiple*.

En este esquema hay tres operaciones de bloqueo:

- bloquear_lectura(X)
- bloquear_escritura(X)
- desbloquear(X)

Ahora, un candado asociado a un elemento X , $CANDADO(X)$ tiene tres posibles estados:

- Bloqueado para leer.
- Bloqueado para escribir.
- Desbloqueado.

Un elemento **bloqueado para leer** posee un candado compartido, porque otras transacciones pueden leer el elemento, en cambio, un elemento **bloqueado para escribir** posee un candado exclusivo, porque una sola transacción posee de manera exclusiva el candado del elemento.

Cada candado puede ser un registro con tres campos: <nombre del elemento de información, CANDAW, núm_de_lecturas >. El valor de CANDADO es un código apropiado para representar uno de los tres estados mencionados anteriormente.

Cuando usamos el esquema de *bloqueo de modo múltiple*, el sistema debe hacer cumplir las siguientes reglas:

1. Una transacción T debe emitir la operación bloquear_lectura(X) o bloquear_escritura(X) antes de que se realice cualquier operación leer-elemento (X) de T .
2. Una transacción T debe emitir la operación bloquear_escritura (X) antes de que se realice cualquier operación escribir_elemento(X) de T .
3. Una transacción T debe emitir la operación desbloquear (X) una vez que se hayan completado todas las operaciones leer_elemento(X) y escribir_elemento(X) de T .
4. Una transacción T no emitirá una operación bloquear_lectura(X) si ya posee un candado de lectura (compartido) o de escritura (exclusivo) para el elemento X .
5. Una transacción T no emitirá una operación bloquear_escritura(X) si ya posee un candado de lectura (compartido) o de escritura (exclusivo) para el elemento X .
6. Una transacción T no emitirá una operación desbloquear(X) a menos que ya posea un candado de lectura (compartido) o de escritura (exclusivo) para el elemento X .

En ocasiones es deseable permitir excepciones a las condiciones 4 y 5 de la lista anterior.

```

bloquear_lectura(X):
  B: si CANDADO(X)="desbloqueado"
    entonces comenzar CANDADO(X) ← "bloqueado para leer";
    núm_de_lecturas(X) ← 1
  fin
  de otro modo si CANDADO(X)="bloqueado para leer"
    entonces núm_de_lecturas(X) ← núm_de_lecturas(X) + 1
    de otro modo comenzar esperar (hasta que CANDADO(X)="desbloqueado"
      y el gestor de bloqueo despierte la transacción);
    ir a B
  fin;

bloquear_escritura(X):
  B: si CANDADO(X)="desbloqueado"
    entonces CANDADO(X) ← "bloqueado para escribir"
  de otro modo comenzar
    esperar (hasta que CANDADO(X)="desbloqueado" y
      el gestor de bloqueo despierte la transacción);
  ir a B
  fin;

desbloquear(X):
  si CANDADO(X)="bloqueado para escribir"
    entonces comenzar CANDADO(X) ← "desbloqueado";
    despertar una de las transacciones que esperan, si las hay
  fin
  de otro modo si CANDADO(X)="bloqueado para leer"
    entonces comenzar
      núm_de_lecturas(X) ← núm_de_lecturas(X) - 1;
      si núm_de_lecturas(X)=0
        entonces comenzar CANDADO(X) ← "desbloqueado";
        despertar una de las transacciones que esperan, si las hay
      fin
    fin;

```

Figura 2. Operaciones de bloqueo y desbloqueo para candados de dos modos (leer_escritura o compartido_exclusivo) [3].

2.3 Conversión de bloqueos

Cuando una transacción solicita un bloqueo de un modo particular sobre un elemento de datos y ninguna otra transacción posee un bloqueo sobre el mismo elemento de datos en un modo conflictivo, se puede conceder el bloqueo. Sin embargo, hay que tener cuidado para evitar la siguiente situación:

Supóngase que la transacción T_2 posee un bloqueo en modo compartido sobre un elemento de datos y que la transacción T_1 solicita un bloqueo en modo exclusivo sobre dicho elemento de datos. Obviamente T_1 debe esperar a que T_2 libere el bloqueo en modo compartido.

Mientras tanto, la transacción T_3 puede solicitar un bloqueo en modo compartido sobre el mismo elemento de datos. La petición de bloqueo es compatible con el bloqueo que se ha concedido a T_2 , por lo tanto, se puede conceder a T_3 el bloqueo en modo compartido. En este punto T_2 puede liberar el bloqueo, pero T_1 debe seguir esperando hasta que T_3 termine. Además, puede haber una nueva transacción T_4 que solicite un bloqueo en modo compartido sobre el mismo elemento de datos y a la cual se conceda el bloqueo antes de que T_3 lo libere.

De hecho, es posible que haya una secuencia de transacciones que soliciten un bloqueo en modo compartido sobre el elemento de datos, y que cada una de ellas libere el bloqueo un poco después de que sea concedido, de forma que T_1 nunca obtenga el bloqueo en modo exclusivo sobre el elemento de datos. La transacción T_1 nunca progresa y se dice que tiene **inanición**.

Se puede evitar la **inanición** de las transacciones al conceder los bloqueos de manera que, cuando una transacción T_i solicite un bloqueo sobre un elemento de datos Q en un modo particular M , el gestor de control de concurrencia conceda el bloqueo siempre que:

1. No exista otra transacción que posea un bloqueo sobre Q en un modo que esté en conflicto con M .
2. No exista otra transacción que esté esperando un bloqueo sobre Q y que lo haya solicitado antes que T_i .

De este modo, una petición de bloqueo nunca se quedará bloqueada por otra petición de bloqueo solicitada más tarde.

2.4 Protocolo de bloqueo de dos fases

Se dice que una transacción sigue el protocolo de **bloqueo de dos fases** si todas las operaciones de bloqueo (bloquear_lectura, bloquear_escritura) preceden a la primera operación [3].

Este protocolo asegura la secuencialidad. El mismo exige que cada transacción realice las peticiones de bloqueo y desbloqueo de dos fases [6]:

1. **Fase de crecimiento**. Una transacción puede obtener bloqueos, pero no puede liberarlos.
2. **Fase de decrecimiento**. Una transacción puede liberar bloqueos pero no puede obtener ninguno nuevo.

Inicialmente una transacción está en la fase de crecimiento. La transacción adquiere los bloqueos que necesite. Una vez que la transacción libera un bloqueo, entra en la fase de decrecimiento y no puede realizar más peticiones de bloqueo [6].

El bloqueo de dos fases puede limitar el grado de concurrencia que pueda haber en un plan. La razón es que, una transacción T tal vez no pueda liberar un elemento X cuando termine de usarlo si T debe bloquear un elemento adicional Y más adelante, o bien, T deberá bloquear el elemento adicional Y antes de necesitarlo para poder liberar X .

Así, X debe permanecer bloqueado por T hasta que ésta haya bloqueado todos los elementos que va a necesitar; sólo entonces podrá T liberar X . Mientras tanto, otra transacción que desee tener acceso a X podría verse obligada a esperar, aunque T ya haya terminado de usarlo, o bien, si Y queda bloqueado antes de que se le necesite realmente, otra transacción que desee tener acceso a él se verá obligada a esperar, aunque T todavía no esté usando Y [3].

Se puede mostrar que el protocolo de bloqueo de dos fases asegura la secuencialidad en cuanto a conflictos. Considérese cualquier transacción, el punto de la planificación en el cual la transacción obtiene su bloqueo final (el final de la fase de crecimiento) se denomina *punto de bloqueo* de la transacción. Ahora, se pueden ordenar las transacciones en base a sus puntos de bloqueo, de hecho, esta ordenación es un orden de secuencialidad para las transacciones [6].

Otra variante del bloqueo de dos fases es el **protocolo de bloqueo riguroso de dos fases**, el cual, exige que se posean todos los bloqueos hasta que se comprometa la transacción. Se puede comprobar fácilmente que con este tipo de bloqueo se pueden secuenciar las transacciones en el orden en que se comprometen. Muchos sistemas de bases de datos implementan tanto el bloqueo estricto como el bloqueo estricto de dos fases [6].

2.5 Interbloqueos e inanición

El **interbloqueo** se produce cuando cada transacción T en un conjunto de dos o más transacciones está esperando a algún elemento que está bloqueado por alguna otra transacción T de dicho conjunto [2]. Por el cual, las transacciones del conjunto están paradas en modo de espera a que alguna de las demás transacciones del conjunto libere el bloqueo que posee sobre un elemento.

2.6 Protocolos de prevención de interbloqueos

Un modo de evitar el interbloqueo es por medio de este protocolo que se utiliza en el bloqueo en dos fases conservador. Requiere que cada transacción bloquee con antelación todos los elementos que necesita (algo que normalmente no es un supuesto práctico), pero como inconveniente si alguno de los elementos no puede obtenerse, ninguno de los elementos se bloquea. En cambio, la transacción espera y después intenta de nuevo bloquear todos los elementos que necesita, lo cual limita la concurrencia.

Otro protocolo conocido que también limita la concurrencia, consiste en ordenar todos los elementos de la base de datos y asegurarse de que, si una transacción necesita de varios elementos, éstos serán bloqueará según ese orden. Lo cual como inconveniente obliga al programador o al sistema, a tener conocimiento el orden de los elementos, motivo por el cual tampoco resulta práctico.

Buscando una solución, se propusieron otros esquemas de prevención del interbloqueo:

Unas técnicas utilizan el concepto de marca de tiempo de transacción, que es un identificador único asignado a cada transacción. Hay dos esquemas de prevención del interbloqueo, denominados esperar_morir y herir_esperar.

En esperar_morir, una transacción más antigua tiene que esperar a una transacción más nueva, mientras que una transacción más nueva que solicita un elemento que posee una transacción más antigua es abortada y reiniciada.

En herir_esperar, una transacción más nueva tiene que esperar a una transacción más antigua, mientras que una transacción más antigua que solicita un elemento que posee una transacción más nueva se apropia de la transacción más nueva abortándola.

Otro grupo de protocolos que evitan el interbloqueo no requiere las marcas de tiempo, son los algoritmos de no espera y espera cautelosa.

En el algoritmo de **no espera**, si una transacción no puede lograr un bloqueo, aborta y se reinicia tras un lapso de tiempo, sin comprobar si se ha producido o no un interbloqueo. Como este esquema puede provocar que las transacciones aborten y reinicien innecesariamente, se propuso el algoritmo de espera cautelosa para intentar reducir el número de abortos/reinicios innecesarios.

En espera cautelosa, si una transacción T_i no está bloqueada (no está esperando por algún otro elemento bloqueado), entonces la transacción T_i es bloqueada y puede esperar; en caso contrario, T_i aborta.

2.7 Detección del interbloqueo

Esta metodología es más práctica para tratar con el interbloqueo. Consiste en que el sistema comprueba si se produjo un estado de interbloqueo. Esta solución resulta atractiva siempre y cuando se espera que haya poca interferencia entre las transacciones.

Normalmente sucede si se trata de transacciones cortas y en donde cada una sólo bloquea unos cuantos elementos, o en el caso de que la carga de transacciones sea ligera. Por el contrario, si las transacciones son largas y utilizan muchos elementos, o si la carga de la transacción es demasiado grande, puede ser óptimo utilizar un esquema de prevención del interbloqueo.

Una forma sencilla de detectar un estado de interbloqueo es mediante un gráfico de espera. En este gráfico se crea un nodo por cada transacción que actualmente se está ejecutando.

Tenemos un estado de interbloqueo si, y sólo si, el gráfico de espera tiene un ciclo. Si el sistema se encuentra en estado de interbloqueo, algunas de las transacciones que lo provocan deben abortarse.

2.8 Tiempos limitados

Una solución muy sencilla es el uso de tiempos limitados. El método consiste en que, si una transacción se encuentra en espera un tiempo superior a un tiempo definido por el sistema, el sistema asume que la transacción puede estar bloqueada y procede a su eliminación (independientemente de que exista o no un interbloqueo).

2.9 Inanición (espera indefinida)

Otro problema que puede surgir cuando se utiliza el bloqueo es la inanición, que tiene lugar cuando una transacción no puede continuar durante un periodo de tiempo indeterminado mientras otras transacciones del sistema continúan con normalidad. Esto puede ocurrir si el esquema de espera para los elementos bloqueados es injusto, dando prioridad a algunas transacciones sobre otras. Una solución es tener un esquema de espera justo, como el uso de una cola del tipo primero en llegar, primero en ser servido.

3. Control de la concurrencia basado en la ordenación de marcas de tiempo.

Dada una transacción que necesita un elemento que ya está bloqueado, puede verse obligada a esperar de manera indefinida hasta que el elemento que está solicitando sea liberado. Una metodología diferente al de bloqueos que garantiza la serialización, implica el uso de marcas de tiempo para ordenar la ejecución de las transacciones para una planificación en serie equivalente.

3.1 Marcas de tiempo

Una **marca de tiempo** es un identificador único creado por el DBMS para identificar una transacción. Normalmente, los valores de las marcas de tiempo son asignados en el orden en el que las transacciones son enviadas al sistema.[2]

Las marcas de tiempo se generan de varias formas. Una posibilidad, es el uso de un contador que se vaya incrementando en uno cada vez que su valor es asignado a una transacción. Las marcas de tiempo se enumeran de forma sencilla de la manera 1, 2, 3, ..., n, en este esquema. Un contador tiene un valor máximo finito, lo que obliga al sistema a reiniciar periódicamente el contador a cero cuando no se ejecutan transacciones durante un corto periodo de tiempo.

3.1.1 Algoritmo de ordenación de marcas de tiempo

Como idea central para este esquema, se desea ordenar las transacciones según sus marcas de tiempo. Una planificación en la que participan las marcas de tiempo es serializable, y la planificación en serie equivalente posibilitará tener las transacciones ordenadas según sus marcas de tiempo. Esto es lo que se conoce como **ordenación por marcas de tiempo**.

El algoritmo debe garantizar que, para cada elemento accedido por operaciones en conflicto de la planificación, el orden con el que se accede a ese elemento no viola el orden de serialización.

3.1.2 Ordenación de marcas de tiempo (TO, Timestamp Ordering) básica.

Siempre que alguna transacción intenta emitir una operación leer_elemento(X) o escribir_elemento(X), el algoritmo *TO* básico compara la marca de tiempo de la transacción con marca_lectura(X) y marca_escritura(X) para garantizar que no se viola el orden de las marcas de tiempo de la transacción en ejecución. Si se viola este orden, la transacción se cancela y se vuelve a enviar al sistema como una transacción nueva con una marca de tiempo nueva.

En el caso de que la transacción T sea abortada y anulada, también debe anularse cualquier transacción anterior T_1 que pueda haber utilizado un valor escrito por T . De forma parecida, para las transacciones anteriores que pueda haber utilizado un valor escrito por T . Es un efecto que se conoce como ***anulación en cascada***.

3.1.3 Ordenación de marcas de tiempo (TO, Timestamp Ordering) estricta

Es una variante de la TO básica que garantiza que las planificaciones son tanto estrictas (para una recuperabilidad fácil) como serializables (por conflicto).

Para implementar este algoritmo, es necesario simular el bloqueo de un elemento X que ha sido escrito por la transacción hasta que dicha transacción es confirmada o abortada. Como consecuencia de este control previo por medio de una simulación, el algoritmo no provoca el interbloqueo.

4. Técnicas multiversión para controlar la concurrencia

Otros protocolos de control de la concurrencia se basan en conservar los valores antiguos de un elemento de datos cuando es actualizado. Es lo que se conoce como **control multiversión de la concurrencia**, porque se conservan varias versiones (valores) de un elemento. Cuando una transacción necesita acceder a un elemento, se elige una versión adecuada para mantener la serialización de la planificación actualmente en ejecución, si es posible.

Debido a que debe ser capaz de almacenar varias versiones de la base de datos, puede resultar en un importante inconveniente de espacio. Por otra parte, es necesario conservar de todos modos versiones antiguas de la base de datos con el fin de realizar operaciones sobre datos antiguos como, por ejemplo, el de la recuperación de datos. Una alternativa también es una base de datos temporal, que hace un seguimiento de los cambios y horarios en que se produjeron. Puesto que ya tiene las versiones antiguas guardadas.

4.1 Técnica multiversión basada en la ordenación de las marcas de tiempo

En este método, el sistema guarda varias versiones de cada elemento de datos. Por cada versión, se conservan el valor de versión y estas dos marcas de tiempo:

marca_lectura(X): La marca de tiempo de lectura es el valor más grande de las marcas de tiempo de todas las transacciones que han leído satisfactoriamente la versión del elemento.

marca_escritura(X): La marca de tiempo de escritura es la marca de tiempo de la transacción que escribió el valor de la versión del elemento.

4.2 Bloqueo en dos fases multiversión utilizando bloqueos de certificación

En este esquema existen tres modos de bloqueo para un elemento: lectura, escritura y certificación. Por tanto, el estado de *BLOQUEAR(X)* para un elemento X puede ser de tres tipos: *bloqueado para lectura*, *bloqueado para escritura*, *bloqueado para certificación* o *desbloqueado*.

La descripción de las relaciones entre los bloqueos de lectura y escritura del esquema estándar (en el cual un bloqueo de escritura era exclusivo) se realiza mediante una tabla de compatibilidad de bloqueos.

Una entrada "Sí" significa que si una transacción T posee el tipo de bloqueo especificado en la cabecera de la columna del elemento X y si la transacción T solicita el tipo de bloqueo especificado en la cabecera de fila del mismo elemento X , entonces T puede obtener el bloqueo porque los modos de bloqueo son compatibles.

De modo contrario, una entrada "No" en la tabla indica que los bloqueos son incompatibles, por lo que T debe esperar hasta que T libere el bloqueo.

	Lectura	Escritura
Lectura	Sí	No
Escritura	No	No

Figura 3. Tabla de compatibilidad para el esquema de bloqueo [3].

La idea es permitir que otras transacciones puedan leer un elemento mientras una transacción posee un bloqueo de escritura sobre ese mismo elemento. Para conseguir esto, es necesario mantener dos versiones de cada elemento X , una de ellas deberá haber sido escrita por alguna transacción confirmada.

La segunda versión X' se crea cuando una transacción T adquiere un bloqueo de escritura sobre el elemento. Otras transacciones pueden continuar leyendo la versión confirmada de X mientras T conserva el bloqueo de escritura.

La transacción T puede escribir el valor de X' cuando lo necesite, sin que ello afecte al valor de la versión confirmada de X . Sin embargo, una vez que T está confirmada, debe obtener un bloqueo de certificación sobre todos los elementos sobre los que actualmente tiene bloqueos de escritura antes de poder confirmarse.

El bloqueo de certificación no es compatible con los bloqueos de lectura, por lo que la transacción puede que tenga que retrasar su confirmación hasta que todos sus elementos bloqueados para escritura sean liberados por cualquier transacción de lectura a fin de obtener los bloqueos de certificación. Una vez adquiridos estos bloqueos (que son bloqueos exclusivos), la versión confirmada de X del elemento de datos se establece al valor de la versión X' , la versión X' es descartada y los bloqueos de certificación son liberados.

	Lectura	Escritura	Certificación
Lectura	Sí	Sí	No
Escritura	Sí	No	No
Certificación	No	No	No

Figura 4. Tabla de compatibilidad para el esquema de bloqueo de lectura/escritura/certificación [3].

5. Técnicas de control de la concurrencia optimistas (validación)

En las técnicas vistas hasta ahora, se realiza un control de la concurrencia antes de que se pueda ejecutar una operación sobre la base de datos. Esta comprobación genera un coste durante la ejecución de la transacción.

En las técnicas de control de la concurrencia optimista no se realiza ninguna comprobación mientras la ejecución se está ejecutando.

Durante la ejecución de la transacción, todas las actualizaciones se aplican a copias locales de los elementos de datos que se conservan para la transacción. Al término de la ejecución de la transacción, una fase de validación comprueba si alguna de las actualizaciones de la transacción viola la serialización.

El sistema debe conservar cierta información necesaria para la fase de validación. Si la serialización no se viola, la transacción es confirmada y se actualiza la base de datos a partir de las copias locales. En caso contrario, la transacción es abortada y reiniciada posteriormente. [2]

Este protocolo tiene tres fases bien definidas:

Fase de lectura. Una transacción puede leer distintos valores de los elementos de datos confirmados de la base de datos. Sin embargo, las actualizaciones sólo se aplican a las copias locales (versiones) de los elementos de datos conservados en el espacio de trabajo de la transacción.

Fase de validación. Orientado a garantizar que la serialización no será violada si las actualizaciones de la transacción se aplican a la base de datos.

Fase de escritura. Dada la fase de validación de forma satisfactoria, recién entonces las actualizaciones de la transacción son aplicadas a la base de datos, en caso contrario, las actualizaciones son descartadas y se reinicia la transacción.

La idea tras el control optimista de la concurrencia es realizar las comprobaciones de inmediato, de manera que la ejecución de la transacción procede con un coste mínimo hasta alcanzar la fase de validación.

6. Granularidad de los elementos de datos y bloqueo de la granularidad múltiple

Consideraciones sobre el nivel de granularidad para el bloqueo:

El tamaño de los elementos de datos se denomina con frecuencia **granularidad** del elemento de datos. La **granularidad fina** se refiere a los tamaños de elemento pequeños, mientras que la **granularidad gruesa** se refiere a los tamaños de elemento más grandes.

Es importante elegir bien el tamaño de los elementos de datos. En primer lugar, se puede observar que, cuanto mayor es el tamaño del elemento de datos, más bajo es el grado de concurrencia permitido. Como consecuencia, cuanto más pequeño es el tamaño del elemento de datos, mayor número de elementos hay en la base de datos.

Como cada elemento está asociado con un bloqueo, el sistema tendrá una cantidad mayor de bloqueos activos que el gestor de bloqueos deberá controlar. Se realizarán más operaciones de bloqueo y desbloqueo, lo que provocará una sobrecarga más alta.

También, se necesitará más espacio para el almacenamiento de la tabla de bloqueos, y lo mismo para el caso de las marcas de tiempo, donde se necesita almacenar las marcas de lectura y de escritura de cada elemento de datos.

Por lo general se consideran los siguientes criterios para el tamaño del elemento:

- Si una transacción típica accede a pocos registros, lo mejor es tener una granularidad equivalente a un registro.
- Si una transacción accede normalmente a muchos registros del mismo fichero, lo mejor es tener una granularidad equivalente a un bloque o un fichero, de modo que la transacción considerará todos los registros como uno (o unos cuantos) elementos de datos.

6.1 Bloqueo por nivel de granularidad múltiple

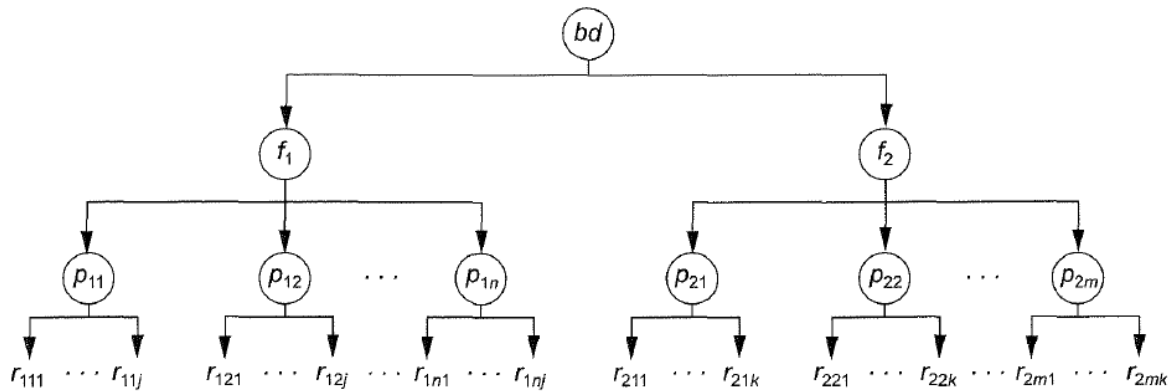


Figura 5. Protocolo de nivel de granularidad múltiple [3].

Ya que el mejor tamaño de granularidad depende de la transacción dada, parece adecuado que un sistema de bases de datos deba soportar varios niveles de granularidad.

La Figura 5 muestra una sencilla jerarquía de granularidad con una base de datos que contiene dos ficheros, cada uno de ellos con varias páginas de disco, y cada página con varios registros.

Esto puede utilizarse para ilustrar un protocolo de nivel de granularidad múltiple, donde es posible que se solicite un bloqueo a cualquier nivel.

Para que resulte práctico el bloqueo por nivel de granularidad múltiple, son necesarios otros tipos de bloqueos, denominados bloqueos de intención. Lo que se busca es que una transacción indique, junto con la ruta desde la raíz hasta el nodo deseado, el tipo de bloqueo (compartido o exclusivo) que requerirá de uno de los descendientes del nodo. Hay tres tipos de bloqueos de intención:

1. **Bloqueo de intención compartida (IS).** Indica que se solicitará un(os) bloqueo(s) compartido(s) en algún(os) (de los) nodo(s) descendiente(s).
2. **Bloqueo de intención exclusiva (IX).** Indica que se solicitará un(os) bloqueo(s) exclusivo(s) en algún(os) (de los) nodo(s) descendiente(s).
3. **Bloqueo compartido con intención exclusiva (SIX).** Indica que el nodo actual está bloqueado en modo compartido, pero se solicitará un(os) bloqueo(s) exclusivo(s) en algún(os) (de los) nodo(s) descendiente(s).

7. Uso de bloqueos para controlar la concurrencia en los índices.

El bloqueo en dos fases también se puede aplicar a los índices, donde los nodos de un índice son equiparables a las páginas de disco. Sin embargo, la posesión de bloqueos en las páginas de índice hasta la fase de reducción de *2PL* puede provocar una cantidad indebida de bloqueos de transacciones porque la búsqueda en un índice siempre empieza por la raíz.

Por consiguiente, si una transacción quiere insertar un registro (operación de escritura), se bloqueará la raíz en modo exclusivo, para que todas las demás solicitudes de bloqueo en conflicto para el índice deban esperar hasta que la transacción entre en la fase de reducción. Esto, bloquea el acceso al índice a todas las demás transacciones, por lo que en la práctica debemos utilizar otras metodologías para bloquear un índice.

La estructura de árbol del índice puede beneficiarse de cuando se desarrolla un esquema de control de la concurrencia.

Por ejemplo, cuando se está ejecutando una búsqueda en un índice (operación de lectura), se recorre el árbol desde la raíz hasta una hoja. Una vez que se ha accedido a un nodo del nivel inferior, los nodos del nivel superior de esa ruta no se utilizarán de nuevo. Así, una vez obtenido un bloqueo de lectura sobre un nodo hijo, se puede liberar el bloqueo sobre el padre. Cuando se está aplicando una inserción a un nodo hoja (es decir, cuando se insertan una clave y un puntero), entonces, debe bloquearse un nodo hoja específico en modo exclusivo.

Un método más conservador para las inserciones sería bloquear el nodo raíz en modo exclusivo y después acceder al nodo hijo apropiado de la raíz. Si el nodo hijo no está lleno, entonces el bloqueo sobre el nodo raíz puede liberarse. Este método puede aplicarse hasta llegar a la hoja, lo que normalmente suponen tres o cuatro niveles desde la raíz. Aunque se posean los bloqueos exclusivos, se liberan pronto.

Una alternativa, el método más optimista, sería solicitar y mantener bloqueos compartidos sobre los nodos que llevan hasta el nodo hoja, con un bloqueo exclusivo sobre esa hoja. Si la inserción provoca la división de la hoja, la inserción se propagará al o los nodos del nivel superior. Después, los bloqueos sobre el o los nodos del nivel superior pueden actualizarse al modo exclusivo.

Otro método para bloquear un índice es utilizar una variante del árbol *B +*, denominada árbol de enlace *B*. En este tipo de árbol, los nodos hermanos del mismo nivel se enlazan en cada nivel. Esto, permite el uso de bloqueos compartidos al solicitar una página, y requiere que el bloqueo sea liberado antes de acceder al nodo hijo.

En una operación de inserción, el bloqueo compartido sobre un nodo se actualizaría al modo exclusivo. Si se produce una división, hay que volver a bloquear el nodo padre en modo exclusivo.

Hay una complicación en las operaciones de búsqueda ejecutadas concurrentemente con la actualización. Suponga que una operación de actualización concurrente sigue la misma ruta que la búsqueda, e inserta una entrada nueva en el nodo hoja. Además, suponga que la inserción provoca la división del nodo hoja. Cuando la inserción se ha realizado, se reanuda el proceso de búsqueda, siguiendo el puntero hasta la hoja deseada, sólo para ver que la clave que se está buscando no está presente porque la división la ha movido a un nodo hoja nuevo, que sería el hermano derecho del nodo hoja original. Sin embargo, el proceso de búsqueda todavía puede tener éxito si sigue el puntero (enlace) desde el nodo hoja original hasta su hermano derecho, adonde se habrá movido la clave deseada.

La manipulación de la eliminación, donde se combinan dos o más nodos del árbol de índice, también forma parte del protocolo de concurrencia de árbol de enlace *B*. En este caso, se mantienen los bloqueos sobre los nodos que se van a combinar, así como un bloqueo sobre el padre de los dos nodos a combinar.

8. Otros problemas del control de la concurrencia

8.1 Inserción, eliminación y registros fantasmas

Cuando se quiere insertar un nuevo elemento a una base de datos, no existen maneras de acceder a él por lógicas razones de que no se ha completado su inserción. Es por eso que un permiso exclusivo de escritura en un entorno de bloqueo, no tendría inconveniente alguno lo mismo para los demás tipos de protocolos de control de concurrencia.

Sin embargo, si consideramos la operación de eliminación a un elemento de datos existentes. En los protocolos de bloqueo, de nuevo hay que obtener un bloqueo exclusivo (escritura) antes de que la transacción pueda eliminar el elemento. Para la ordenación de las marcas de tiempo, el protocolo debe garantizar que ninguna transacción posterior ha leído o escrito el elemento antes de permitir la eliminación del elemento. [2]

La situación conocida como problema del fantasma se da cuando se está insertando un registro nuevo que de una transacción T que satisface una condición que un conjunto de registros accedido por otra transacción T' tiene acceso.

Puede ocurrir el llamado problema del fantasma cuando un registro que está siendo insertado por alguna transacción T satisface una condición que deberá satisfacer un conjunto de registros a los que tenga acceso otra transacción T' debe satisfacer.

Como ejemplo, supongamos que la transacción T está insertando un nuevo registro *DOCENTE* cuyo número de legajo es igual a 5 (el campo *id_legajo* es igual a 5), y que la transacción T' está teniendo acceso a todos los registros *DOCENTE* y se desee sumar todos sus valores Sueldo. Si el orden en serie equivalente es T seguida por T' , entonces T debe leer el registro *DOCENTE* nuevo e incluir su Sueldo en la suma. Si el orden es T' seguida por T , no se incluiría el sueldo nuevo.

8.2 Transacciones interactivas

Un problema muy normal se produce cuando las transacciones interactivas leen la entrada y escriben la salida en dispositivos interactivo como monitores antes de ser confirmadas.

En realidad, el problema surge desde el usuario, porque es él quien puede introducir un valor desde para un elemento de datos en una transacción T , basándose en el valor que puede visualizarse desde la pantalla por la transacción T' , que puede que no esté confirmada.

Una manera de tratar este problema sería en posponer la salida de las transacciones a pantalla hasta que las transacciones hayan sido confirmadas.

8.3 Cerrojos

Se los denomina **cerrojos** a los bloqueos que se mantienen durante poco tiempo y como característica especial, no obedecen el protocolo de control de la concurrencia usual. Por este motivo puede utilizarse para garantizar la integridad física de, por ejemplo, una página cuando se está grabando de la memoria al disco y, finalmente, se liberaría el cerrojo.

9. Conclusión

Los protocolos basados en los bloqueos son los más utilizados en la práctica. Realizamos una introducción de los conceptos de bloqueos compartidos y exclusivos, y observamos cómo el bloqueo puede garantizar la serialización al utilizarse en combinación con la regla de bloqueo en dos fases.

También, presentamos varias técnicas para tratar con el problema del interbloqueo, que puede surgir con los bloqueos. Entre ellos, encontramos el protocolo de ordenación de marcas de tiempo, que garantiza la serialización basada en la ordenación de las marcas de tiempo de las transacciones.

También hemos visto el protocolo de ordenación de marcas de tiempo estricto. Una técnica, denominada bloqueo en dos fases multiversión, asume que pueden existir dos versiones de un elemento e intenta aumentar la concurrencia haciendo compatibles los bloqueos de escritura y lectura.

También, hemos presentado un protocolo multiversión basado en la ordenación de las marcas de tiempo, y un ejemplo de protocolo optimista, que también se conoce como protocolo de certificación o validación. Describimos un protocolo de bloqueo multigranularidad que permite modificar la granularidad en base a la mezcla de transacciones del momento, con el objetivo de mejorar el rendimiento del control de la concurrencia.

Por último, hablamos del problema del fantasma y de los problemas relacionados con las transacciones interactivas, y describimos brevemente el concepto de cerrojos y de sus diferencias con los bloqueos.

Bibliografía:

- [1] Guillermo Arduino, Pedro L. Alfonzo. *Técnicas de control de concurrencia en base de datos: implementación en un sistema de gestión*. Corrientes, Argentina: Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste, 2018.
- [2] Universidad de Valencia, "Técnicas de control de concurrencia". 2006/07. [En línea]. Disponible: http://informatica.uv.es/iiguia/2000/BD2/4_0_BD2Tema4_06.pdf
- [3] Elmasri, Ramez y Navathe, Shamkant B. "Fundamentos de sistemas de base de datos". 5ta Edición. Ed. Pearson.
- [4] Microsoft, "Guía de versiones de fila y bloqueo de transacciones de SQL Server". Septiembre 2015. [En línea]. Disponible: [https://docs.microsoft.com/es-mx/previous-versions/sql/sql-server-guides/jj856598\(v=sql.110\)?redirectedfrom=MSDN](https://docs.microsoft.com/es-mx/previous-versions/sql/sql-server-guides/jj856598(v=sql.110)?redirectedfrom=MSDN)
- [5] Silberschatz, A., Korth, F., Sudarshan, S. *Fundamentos de Base de Datos*, Madrid: McGraw-Hill/Interamericana De España, S. A. U., 2002.